

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

“Введение в СУБД MongoDB. Установка MongoDB. Работа с БД в
СУБД MongoDB”

Обучающийся Артемов Илья Витальевич

Группа K3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2026

ЧАСТЬ 1 — УСТАНОВКА MONGODB

brew tap mongodb/brew

brew install mongodb-community

brew services start mongodb-community

mongosh

```
goodkid@MacBook-Pro-Ila-3 ~ % mongosh
Current Mongosh Log ID: 6a1ada8fd3a5fd4b1f01c11d
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.3.3
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
  The server generated these startup warnings when booting
  2026-05-30T15:39:30.334+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  -----

test> 
```

ЛР 6.1 — БАЗОВЫЕ ОПЕРАЦИИ

Задание 3 — Методы db

db.help() - список методов объекта db

```
test> db.help()

Database Class:

getMongo                Returns the current database connection
getName                 Returns the name of the DB
getCollectionNames      Returns an array containing the names of all collections in the current database.
getCollectionInfos      Returns an array of documents with collection information, i.e. collection name and options, for the current database.
runCommand              Runs an arbitrary command on the database.
adminCommand            Runs an arbitrary command against the admin database.
aggregate               Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB            Returns another database without modifying the db variable in the shell environment.
getCollection            Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
dropDatabase            Removes the current database, deleting the associated data files.
createUser              Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
updateUser              Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword      Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
logout                  Ends the current authentication session. This function has no effect if the current session is not authenticated.
dropUser                Removes the user from the current database.
dropAllUsers            Removes all users from the current database.
auth                    Allows a user to authenticate to the database from within the shell.
grantRolesToUser        Grants additional roles to a user.
revokeRolesFromUser     Removes a one or more roles from a user on the current database.
getUsers                Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
getUsers                Returns information for all the users in the database.
createCollection         Create new collection
createEncryptedCollection Creates a new collection with a list of encrypted fields each with unique and auto-created data encryption keys (DEKs). This is a utility function that internally utilises ClientEncryption.cr
eateEncryptedCollection.
createView              Create new view
createRole              Creates a new role.
updateRole              Updates the role's profile on the database on which you run the method. An update to a field completely replaces the previous field's values.
dropRole               Removes the role from the current database.
dropAllRoles            Removes all roles from the current database.
grantRolesToRole        Grants additional roles to a role.
revokeRolesFromRole     Removes a one or more roles from a role on the current database.
grantPrivilegesToRole   Grants additional privileges to a role.
revokePrivilegesFromRole Removes a one or more privileges from a role on the current database.
getRole                 Returns role information for a specified role. Run this method on the role's database. The role must exist on the database on which the method runs.
getRoles                Returns information for all the roles in the database.
currentOp               Runs an aggregation using $currentOp operator. Returns a document that contains information on in-progress operations for the database instance. For further information, see $currentOp.
killOp                  Calls the killOp command. Terminates an operation as specified by the operation ID. To find operations and their corresponding IDs, see $currentOp or db.currentOp().
shutdownServer           Calls the shutdown command. Shuts down the current mongod or mongos process cleanly and safely. You must issue the db.shutdownServer() operation against the admin database.
fsync                   Calls the fsync command. Forces the mongod to flush all pending write operations to disk and locks the entire mongod instance to prevent additional writes until the user releases the lock wi
th a corresponding db.fsyncLock() command.
fsyncUnlock              Calls the fsyncUnlock command. Reduces the lock taken by db.fsyncLock() on a mongod instance by 1.
version                 returns the db version, uses the buildInfo command
serverBits              returns the db serverBits, uses the buildInfo command
isMaster                Calls the isMaster command
hello                   Calls the hello command
serverBuildInfo          returns the db serverBuildInfo, uses the buildInfo command
serverStatus            returns the server status, uses the serverStatus command
stats                   returns the db stats, uses the dbStats command
hostInfo                Calls the hostInfo command
serverCmdLineOpts        returns the db serverCmdLineOpts, uses the getCmdLineOpts command
rotateCertificates       Calls the rotateCertificates command
printCollectionStats     Prints the collection.stats for each collection in the db.
getProfilingStatus       returns the db getProfilingStatus, uses the profile command

setProfilingLevel        returns the db setProfilingLevel, uses the profile command
setLogLevel              returns the db setLogLevel, uses the setParameter command
getLogComponents         returns the db getLogComponents, uses the getParameter command
cloneDatabase             deprecated, non-functional
cloneCollection           deprecated, non-functional
copyDatabase              deprecated, non-functional
commandHelp              returns the db commandHelp, uses the passed in command with help: true
listCommands             Calls the listCommands command
getLastErrorObj           Calls the getLastError command
getLastError              Calls the getLastError command
printShardingStatus       Calls sh.status(verbose)
printSecondaryReplicationInfo Prints secondary replicaset information
getReplicationInfo        Returns replication information
printReplicationInfo      Formats sh.getReplicationInfo
printSlaveReplicationInfo DEPRECATED. Use db.printSecondaryReplicationInfo
setSecondaryOk            This method is deprecated. Use db.getMongo().setReadPref() instead
watch                     Opens a change stream cursor on the database
sql                       (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an experimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies

test> █
```

db.stats() - статистика текущей БД

```
[test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
test> █
```

Задание 4 — Создать БД learn

use learn

```
[test> use learn
switched to db learn
learn> █
```

Задание 5 — Список доступных БД

show dbs

БД learn не появится в списке пока в ней нет данных


```
[learn> show dbs
admin    40.00 KiB
config   12.00 KiB
local    40.00 KiB
learn> █
```

Задание 6 — Создать коллекцию unicorns

db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})

```
[learn> db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1adb6bd3a5fd4b1f01c11e') }
}
learn> █
```

show dbs

```
}
[learn> show dbs
admin    40.00 KiB
config   12.00 KiB
learn    40.00 KiB
local    40.00 KiB
learn> █
```

Задание 7 — Список коллекций

show collections

```
[learn> show collections
unicorns
learn> █
```

Задание 8 — Переименовать коллекцию

db.unicorns.renameCollection("myunicorns")

show collections

```
[learn> db.unicorns.renameCollection("myunicorns")
{ok: 1}
[learn> show collections
myunicorns
learn>
```

Задание 9 — Статистика коллекции db.myunicorns.stats()

```
learn> db.myunicorns.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4M,app_metadata={formatVersion:1},assert={commit_timestamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=off},block_allocation=best,block_compressor=snappy,block_manager=none,cache_size=0,checksum=false,columns=colgroups,collators=columns,dictionary=disaggregated(page_logs,storage_timestamp),encryption={keyid=none},exclusive=false,extractors={format=bytes,huffman.keys,huffman.values=ignore,in_memory_cache_size=false,immutable=false,import(compare_timestamp=oldest_timestamp,enabled=false,file_metadata=metadata_file,panic_corrupt=true,repair=false),in_memory=false,ingest=internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_max=4M,key_format=key_page=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32M,leaf_value_max=4M,log(enabled=true),low(auto_throttle bloom,bloom_bit_counts,bloom_config,bloom_hash_count,bloom_store_chunk_size,chunk_size=merge_custom(prefix,start_generation=,suffix),merge_max=merge_min),memory_page_image_max=1M,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=98,stable=,tiered_storage={auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0},type=file,value_format=u,value=},write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection-4a29c7b3-b489-4765-8d08-ddf659a5079?',
    version: WiredTiger 12.0.0 (November 15, 2024),
    autocommit: {
      'retires for readonly operations': 0,
      'retires for update operations': 0
    },
    backup: {
      'total modified incremental blocks with compressed data': 0,
      'total modified incremental blocks without compressed data': 0
    },
    'block-disagg': {
      'Bytes read from the shared history store in SLS': 0,
      'Bytes written to the shared history store in SLS': 0,
      'Disaggregated block manager get': 0,
      'Disaggregated block manager get sold page': 0,
      'Disaggregated block manager put from the shared history store in SLS': 0,
      'Disaggregated block manager page discard calls': 0,
      'Disaggregated block manager put': 0,
      'Disaggregated block manager put sold page': 0,
      'Disaggregated block manager put to the shared history store in SLS': 0,
      'Disaggregated block manager read ahead of materialization frontier': 0
    },
    'block-manager': {
      'allocation requiring file extension': 4,
      'blocks allocated': 4,
      'blocks freed': 0,
      'checkpoint size': 4096,
      'file allocation unit size': 4096,
      'file bytes available for reuse': 0,
      'file magic number': 120897,
      'file major version number': 1,
      'file size in bytes': 28408,
      'minor version number': 0
    },
    btrees: {
      'btree checkpoint generation': 0,
      'btree clean tree checkpoint expiration time': Long('9223372036854775807'),
      'btree compact pages reviewed': 0,
      'btree compact pages rewritten': 0,
      'btree compact pages skipped': 0,
      'btree expected number of compact bytes rewritten': 0,
      'btree expected number of compact pages rewritten': 0,
      'btree number of pages reexamined during checkpoint': 2,
      'btree skipped by compaction as process would not reduce size': 0,
      'column-store internal pages': 0,
      'column-store variable-size B+ tree encoded values': 0,
      'column-store variable-size deleted values': 0,
      'column-store variable-size leaf pages': 0,
      'fixed-record size': 0,
      'maximum internal page size': 4096,
      'maximum leaf page key size': 2048,
      'maximum leaf page size': 32768,
      'maximum leaf page value size': 67108864,
      'maximum tree depth': 2,
      'number of key/value pairs': 0,
      'overflow pages': 0,
      'row-store empty values': 0,
      'row-store internal pages': 0,
      'row-store leaf pages': 0
    },
    cache: {
      'application threads eviction requested with cache fill ratio < 25%': 0,
      'application threads eviction requested with cache fill ratio >= 25% and < 50%': 0,
      'application threads eviction requested with cache fill ratio >= 50% and < 75%': 0,
      'application threads eviction requested with cache fill ratio >= 75%': 0,
      'application threads eviction skip page with updates or dirty page': 0,
      'bytes currently in the cache': 1419,
      'bytes dirty in the cache cumulative': 1419,
      'bytes read into cache': 0,
      'bytes written from cache': 141,
      'checkpoint blocked page eviction': 0,
      'checkpoint of history store file blocked non-history store page eviction': 0,
      'data source pages selected for eviction unable to be evicted': 0,
      'dirty internal page cannot be evicted in disaggregated storage': 0,
      'eviction gave up due to detecting a disk value without a timestamp behind the last update on the chain': 0,
      'eviction gave up due to detecting a tombstone without a timestamp ahead of the selected on disk update': 0,
      'eviction gave up due to detecting a tombstone without a timestamp ahead of the selected on disk update after validating the update chain': 0,
      'eviction gave up due to detecting update chain entries without timestamps after the selected on disk update': 0,
      'eviction gave up due to needing to remove a record from the history store but checkpoint is running': 0,
      'eviction gave up due to no progress being made': 0,
      'eviction server skipped the pages when prefetching': 0,
      'eviction walk pages queued that had updates': 0,
      'eviction walk pages queued that were clean': 0,
      'eviction walk pages queued that were dirty': 0,
      'eviction walk pages seen that had updates': 0,
      'eviction walk pages seen that were clean': 0,
      'eviction walk pages seen that were dirty': 0,
      'eviction walk passes of a file': 0,
      'eviction walk target pages histogram - 0-9': 0,
      'eviction walk target pages histogram - 10-21': 0,
      'eviction walk target pages histogram - 128 and higher': 0,
      'eviction walk target pages histogram - 32-43': 0,
      'eviction walk target pages histogram - 44-128': 0,
      'eviction walk target pages reduced due to history store cache pressure': 0,
      'garbage collection from the update drive page is skipped because the prune timestamp has not moved': 0,
      'hazard pointer blocked page eviction': 0,
      'history store cursor not cached during eviction': 0,
      'history store table insert calls': 0,
      'history store table insert calls that returned restart': 0,
      'history store table key to be processed': 0,
      'history store table reads': 0,
      'history store table reads missed': 0,
      'history store table reads requiring squashed modifies': 0,
      'history store table resolved updates without timestamps that lose their durable timestamp': 0,
      'history store table truncation by rollback to stable to remove an unstable update': 0,
      'history store table truncation by rollback to stable to remove an update': 0,
      'history store table truncation to remove all the keys of a btree': 0,
      'history store table truncation to remove an update': 0,
      'history store table truncation to remove range of updates due to an update without a timestamp on data page': 0,
      'history store table truncation to remove range of updates due to key being removed from the data page during reconciliation': 0,
      'history store table truncations that would have happened in non-dryrun mode': 0,
      'history store table truncations to remove an unstable update that would have happened in non-dryrun mode': 0,
      'history store table truncations to remove an update that would have happened in non-dryrun mode': 0,
      'history store table updates to be processed': 0,
      'history store table updates without timestamps fixed up by reinserting with the fixed timestamp': 0,
      'history store table writes requiring squashed modifies': 0,
      'in-memory page passed criteria to be split': 0,
      'in-memory page splits': 0,
      'internal page split blocked its eviction': 0,
      'internal pages evicted': 0,
      'internal pages split during eviction': 0,
      'leaf pages split during eviction': 0,
      'locate a random in-ten ref by examining all entries on the root page': 0,
    }
  }
}
```



```

'modified pages evicted': 0,
'multi-block reconciliation blocked whilst checkpoint is running': 0,
'number of internal pages read that had deltas attached': 0,
'number of leaf pages read that had deltas attached': 0,
'number of times dirty trigger was reached': 0,
'number of times eviction trigger was reached': 0,
'number of times updates trigger was reached': 0,
'number of times when cas update the tree max_lsn failed': 0,
'obsolete updates removed': 0,
'overflow keys on a multiblock row-store page blocked its eviction': 0,
'overflow pages read into cache': 0,
'page eviction blocked due to materialization frontier': 0,
'page eviction blocked in degraded state as it can only be written by the next checkpoint': 0,
'page split during eviction doomed the tree': 0,
'page written requiring history store records': 0,
'pages dirtied due to obsolete time window by eviction': 0,
'pages evicted ahead of the page materialization frontier': 0,
'pages read into cache': 0,
'pages read into cache after truncate': 0,
'pages read into cache after truncate in prepare state': 0,
'pages read into cache by checkpoint': 0,
'pages requested from the cache': 2,
'pages requested from the cache due to pre-fetch': 0,
'pages requested from the cache internal': 0,
'pages requested from the cache leaf': 2,
'pages requested from the history store': 0,
'pages seen by eviction walk': 0,
'pages written from cache': 2,
'pages written requiring in-memory restoration due to invisible updates': 0,
'pages written requiring in-memory restoration due to scrub eviction': 0,
'precise checkpoint caused an eviction to be skipped because any dirty content needs to remain in cache': 0,
'realizing in-memory split after reconciliation failed due to internal lock busy': 0,
'recent modification of a page blocked its eviction': 0,
'reconciled pages scrubbed and added back to the cache clean': 0,
'reverse splits performed': 0,
'reverse splits skipped because of VLCS namespace gap restrictions': 0,
'shared history store cursor not cached during eviction': 0,
'size of delta updates reconstructed on the base page': 0,
'size of tombstones restored when reading a page': 0,
'the number of times full update inserted to history store': 0,
'the number of times reverse modify inserted to history store': 0,
'tracked dirty bytes in the cache': 0,
'tracked dirty internal page bytes in the cache': 0,
'tracked dirty leaf page bytes in the cache': 0,
'uncommitted truncate blocked page eviction': 0,
'unmodified pages evicted': 0
},

```

```

cache_walk: {
  'Average difference between current eviction generation when the page was last considered': 0,
  'Average on-disk page image size seen': 0,
  'Average time in cache for pages that have been visited by the eviction server': 0,
  'Average time in cache for pages that have not been visited by the eviction server': 0,
  'Clean pages currently in cache': 0,
  'Current eviction generation': 0,
  'Dirty pages currently in cache': 0,
  'Entries in the root page': 0,
  'Internal pages currently in cache': 0,
  'Leaf pages currently in cache': 0,
  'Maximum difference between current eviction generation when the page was last considered': 0,
  'Maximum page size seen': 0,
  'Minimum on-disk page image size seen': 0,
  'Number of pages never visited by eviction server': 0,
  'On-disk page image sizes smaller than a single allocation unit': 0,
  'Pages created in memory and never written': 0,
  'Pages currently queued for eviction': 0,
  'Pages that could not be queued for eviction': 0,
  'Refs skipped during cache traversal': 0,
  'Size of the root page': 0,
  'Total number of pages currently in cache': 0
},
checkpoint: { 'checkpoint has acquired a snapshot for its transaction': 0 },
'checkpoint-cleanup': {
  'pages added for eviction': 0,
  'pages dirtied due to obsolete time window': 0,
  'pages read into cache (reclaim_space)': 0,
  'pages read into cache due to obsolete time window': 0,
  'pages removed': 0,
  'pages skipped during tree walk': 0,
  'pages visited': 1
},
compression: {
  'compressed page maximum internal page size prior to compression': 4096,
  'compressed page maximum leaf page size prior to compression': 13107,
  'page written to disk failed to compress': 0,
  'page written to disk was too small to compress': 2,
  'pages read from disk': 0,
  'pages read from disk with compression ratio greater than 64': 0,
  'pages read from disk with compression ratio smaller than 2': 0,
  'pages read from disk with compression ratio smaller than 4': 0,
  'pages read from disk with compression ratio smaller than 8': 0,
  'pages read from disk with compression ratio smaller than 16': 0,
  'pages read from disk with compression ratio smaller than 32': 0,
  'pages read from disk with compression ratio smaller than 64': 0,
  'pages written to disk': 0,
  'pages written to disk with compression ratio greater than 64': 0,
  'pages written to disk with compression ratio smaller than 2': 0,
  'pages written to disk with compression ratio smaller than 4': 0,
  'pages written to disk with compression ratio smaller than 8': 0,
  'pages written to disk with compression ratio smaller than 16': 0,
  'pages written to disk with compression ratio smaller than 32': 0,
  'pages written to disk with compression ratio smaller than 64': 0
},

```

```

cursor: {
  'Total number of deleted pages skipped during tree walk': 0,
  'Total number of entries skipped by cursor next call': 0,
  'Total number of entries skipped by cursor prev call': 0,
  'Total number of entries skipped to position the history store cursor': 0,
  'Total number of in-memory deleted pages skipped during tree walk': 0,
  'Total number of on-disk deleted pages skipped during tree walk': 0,
  'Total number of times a search near has exited due to prefix config': 0,
  'Total number of times cursor fails to temporarily release pinned page to encourage eviction of hot or large page': 0,
  'Total number of times cursor temporarily releases pinned page to encourage eviction of hot or large page': 0,
  'bulk loaded cursor insert calls': 0,
  'cache cursor reuse count': 0,
  'close calls that result in cache': 1,
  'create calls': 2,
  'cursor bound calls that return an error': 0,
  'cursor bounds cleared from reset': 0,
  'cursor bounds comparisons performed': 0,
  'cursor bounds next called on an unpositioned cursor': 0,
  'cursor bounds next early exit': 0,
  'cursor bounds prev called on an unpositioned cursor': 0,
  'cursor bounds prev early exit': 0,
  'cursor bounds search early exit': 0,
  'cursor bounds search near call repositioned cursor': 0,
  'cursor cache calls that return an error': 0,
  'cursor close calls that return an error': 0,
  'cursor compare calls that return an error': 0,
  'cursor equals calls that return an error': 0,
  'cursor get key calls that return an error': 0,
  'cursor get value calls that return an error': 0,
  'cursor insert calls that return an error': 0,
  'cursor insert check calls that return an error': 0,
  'cursor largest key calls that return an error': 0,
  'cursor modify calls that return an error': 0,
  'cursor next calls that return an error': 0,
  'cursor next calls that skip due to a globally visible history store tombstone': 0,
  'cursor next calls that skip greater than 1 and fewer than 100 entries': 0,
  'cursor next calls that skip greater than or equal to 100 entries': 0,
  'cursor next random calls that return an error': 0,
  'cursor prev calls that return an error': 0,
  'cursor prev calls that skip due to a globally visible history store tombstone': 0,
  'cursor prev calls that skip greater than or equal to 100 entries': 0,
  'cursor prev calls that skip less than 100 entries': 0,
  'cursor reconfigure calls that return an error': 0,
  'cursor remove calls that return an error': 0,
  'cursor reopen calls that return an error': 0,
  'cursor reserve calls that return an error': 0,
  'cursor reset calls that return an error': 0,
  'cursor search calls that return an error': 0,
  'cursor search near calls that return an error': 0,
  'cursor update calls that return an error': 0,
  'insert calls': 1,
  'insert key and value bytes': 66,

```

```

modify: 0,
'modify key and value bytes affected': 0,
'modify value bytes modified': 0,
'next calls': 0,
'open cursor count': 0,
'open cursor time application (usecs)': 0,
'open cursor time interval (usecs)': 0,
'operation restarted': 0,
'prev calls': 1,
'remove calls': 0,
'remove key bytes removed': 0,
'reserve calls': 0,
'reset calls': 0,
'search calls': 0,
'search history store calls': 0,
'search near calls': 0,
'truncate calls': 0,
'update calls': 0,
'update key and value bytes': 0,
'update value size change': 0
},
layered: {
'Layered table cursor insert operations': 0,
'Layered table cursor modify operations': 0,
'Layered table cursor next operations': 0,
'Layered table cursor next operations from the ingest btrees': 0,
'Layered table cursor next operations from the stable btrees': 0,
'Layered table cursor prev operations': 0,
'Layered table cursor prev operations from the ingest btrees': 0,
'Layered table cursor prev operations from the stable btrees': 0,
'Layered table cursor remove operations': 0,
'Layered table cursor search near operations': 0,
'Layered table cursor search near operations from the ingest btrees': 0,
'Layered table cursor search near operations from the stable btrees': 0,
'Layered table cursor search operations': 0,
'Layered table cursor search operations from the ingest btrees': 0,
'Layered table cursor search operations from the stable btrees': 0,
'Layered table cursor update operations': 0,
'Layered table cursor upgrade state for the ingest btrees': 0,
'Layered table cursor upgrade state for the stable btrees': 0,
'checkpoints performed on this table by the layered table manager': 0,
'disagg pick up checkpoints failed': 0,
'disagg pick up checkpoints succeeded': 0,
'how many log applications the layered table manager applied on this tree': 0,
'how many log applications the layered table manager skipped on this tree': 0,
'how many previously-applied LQs the layered table manager skipped on this tree': 0
},

```

```

reconciliation: {
'VLCB pages explicitly reconciled as empty': 0,
'approximate byte size of timestamps in pages written': 0,
'approximate byte size of transaction IDs in pages written': 0,
'average length of delta chain on internal page with deltas': 0,
'average length of delta chain on leaf page with deltas': 0,
'changes since prior reconciliation (bucket 1) between 1 and 5': 0,
'changes since prior reconciliation (bucket 2) between 6 and 10': 0,
'changes since prior reconciliation (bucket 3) between 11 and 20': 0,
'changes since prior reconciliation (bucket 4) between 21 and 50': 0,
'changes since prior reconciliation (bucket 5) between 51 and 100': 0,
'changes since prior reconciliation (bucket 6) between 101 and 200': 0,
'changes since prior reconciliation (bucket 7) between 201 and 500': 0,
'changes since prior reconciliation (bucket 8) greater than 500': 0,
'cursor next/prev calls during HS wrapup search_near': 0,
'dictionary matches': 0,
'fast-path pages deleted': 0,
'free page ID due to failed page replacement reconciliation in disagg': 0,
'full internal pages written instead of a page delta': 0,
'full leaf pages written instead of a page delta': 1,
'ingest tree reconciliation keeps the aborted prepare updates': 0,
'internal page delta keys deleted': 0,
'internal page delta keys updated/inserted': 0,
'internal page deltas written': 0,
'internal page key bytes discarded using suffix compression': 0,
'internal page multi-block writes': 0,
'leaf delta page key bytes discarded using prefix compression': 0,
'leaf full page key bytes discarded using prefix compression': 0,
'leaf page deltas written': 0,
'leaf page multi-block writes': 0,
'leaf-page overflow keys': 0,
'max deltas seen on internal page during reconciliation': 0,
'max deltas seen on leaf page during reconciliation': 0,
'maximum blocks required for a page': 1,
'number of keys that are garbage collected from the disk images in the ingest btrees for disaggregated storage': 0,
'number of keys that are garbage collected from the update chains in the ingest btrees for disaggregated storage': 0,
'overflow values written': 0,
'page deltas rejected due to invalid page ID': 0,
'page deltas rejected due to max consecutive limit': 0,
'page deltas rejected due to multiblock reconciliation': 0,
'page deltas rejected due to non-eligible page in previous reconciliation': 0,
'page deltas rejected due to size threshold': 0,
'page deltas rejected due to zero entries': 0,
'page deltas rejected: build function returned false (disabled, in-memory split, or internal page constraints not met)': 0,
'page reconciliation calls': 2,
'page reconciliation calls for eviction': 0,
'page reconciliation calls for pages between 1 and 1000': 0,
'page reconciliation calls for pages between 10 and 10000': 0,
'page reconciliation calls for pages between 10000 and 100': 0,
'page reconciliation calls for pages over 100': 0,
'pages deleted': 0,
'pages eligible for delta generation': 0,
'pages written including an aggregated newest start durable timestamp': 0,
'pages written including an aggregated newest stop durable timestamp': 0,
'pages written including an aggregated newest stop transaction ID': 0,
'pages written including an aggregated newest transaction ID': 0,
'pages written including an aggregated oldest start timestamp': 0,
'pages written including an aggregated prepare': 0,
'pages written including at least one prepare state': 0,
'pages written including at least one start durable timestamp': 0,
'pages written including at least one start timestamp': 0,

```

```

'pages written including an aggregated prepare': 0,
'pages written including at least one prepare state': 0,
'pages written including at least one start durable timestamp': 0,
'pages written including at least one start timestamp': 0,
'pages written including at least one stop durable timestamp': 0,
'pages written including at least one stop timestamp': 0,
'pages written including at least one stop transaction ID': 0,
'pages written with at least one internal page delta': 0,
'pages written with at least one leaf page delta': 0,
'records written including a prepare state': 0,
'records written including a start durable timestamp': 0,
'records written including a start timestamp': 0,
'records written including a stop durable timestamp': 0,
'records written including a stop timestamp': 0,
'records written including a stop transaction ID': 0,
'writes skipped in disaggregated storage': 0
},
session: { 'object compaction': 0 },
transaction: {
'a reader raced with a prepared transaction commit and skipped an update or updates': 0,
'number of times overflow removed value is read': 0,
'race to read prepared update retry': 0,
'rollback to stable applied btrees': 0,
'rollback to stable history store keys that would have been swept in non-dryrun mode': 0,
'rollback to stable history store records with stop timestamps older than newer records': 0,
'rollback to stable inconsistent checkpoint': 0,
'rollback to stable keys removed': 0,
'rollback to stable keys restored': 0,
'rollback to stable keys that would have been removed in non-dryrun mode': 0,
'rollback to stable keys that would have been restored in non-dryrun mode': 0,
'rollback to stable restored timestamps from history store': 0,
'rollback to stable restored updates from history store': 0,
'rollback to stable skipped btrees': 0,
'rollback to stable skipping delete file': 0,
'rollback to stable skipping stable file': 0,
'rollback to stable sweeping history store keys': 0,
'rollback to stable tombstones from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates removed from history store': 0,
'rollback to stable updates that would have been removed from history store in non-dryrun mode': 0,
'update conflicts': 0
},
sharded: false,
size: 40,
count: 1,
numOrphanDocs: 0,
storageSize: 2048,
totalIndexSize: 20480,
totalSize: 4096,
indexSizes: { '_id': 20480 },
avgObjSize: 65,
ns: 'test.mymongo',
nindexes: 1,
scaleFactor: 1
}
leaving

```


Задание 10 — Удалить коллекцию

`db.myunicorns.drop()`

`show collections`

```
[learn> db.myunicorns.drop()
true
[learn> show collections

learn> █
```

Задание 11 — Удалить БД learn

`db.dropDatabase()`

`show dbs`

```
[learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
[learn> show dbs
admin      40.00 KiB
config     60.00 KiB
local      40.00 KiB
learn> █
```

ЛР 6.2 — CRUD ОПЕРАЦИИ

РАЗДЕЛ 2.1 — ВСТАВКА ДАННЫХ

Задание 2.1.1 — Создать БД и заполнить коллекцию

Создаём БД
use learn

Вставляем 11 единорогов

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600,
gender: 'm', vampires: 63})
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender:
'f', vampires: 43})
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984,
gender: 'm', vampires: 182})
db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender:
'm', vampires: 99})
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'],
weight:550, gender:'f', vampires:80})
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733,
gender: 'f', vampires: 40})
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender:
'm', vampires: 39})
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender:
'm', vampires: 2})
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601,
gender: 'f', vampires: 33})
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650,
gender: 'm', vampires: 54})
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender:
'f'})
```

Вставляем одного через переменную

```
document = ({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender:
'm', vampires: 165})
db.unicorns.insert(document)
```

Проверяем коллекцию

db.unicorns.find()

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c11f'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c120'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c121'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c122'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c123'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c124'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
]
```

```

{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c125'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c126'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c127'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c128'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c129'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a1adcffd3a5fd4b1f01c12a'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
]
learn>

```

РАЗДЕЛ 2.2 — ВЫБОРКА ДАННЫХ

Задание 2.2.1

Список самцов, отсортированный по имени:

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
[learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a1adcffd3a5fd4b1f01c12a'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c11f'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c125'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c128'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c126'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c122'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c121'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn> █
```

Список самок, первые 3, отсортированные по имени:

`db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)`

```
[learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c120'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c124'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c127'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> █
```

Самки любящие carrot — через findOne:

`db.unicorns.findOne({gender: 'f', loves: 'carrot'})`

```
[learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c120'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> █
```

Самки любящие carrot — через limit:


```
]
[learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c120'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> █
```

Задание 2.2.2 — Самцы без предпочтений и пола

`db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})`

```

[learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c11f'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c121'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c122'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c125'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c126'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c128'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a1adcffd3a5fd4b1f01c12a'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
learn> █

```

Задание 2.2.3 — Список в обратном порядке добавления

`db.unicorns.find().sort({$natural: -1})`

```

learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a1adcffd3a5fd4b1f01c12a'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c129'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c128'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c127'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c126'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c125'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {

```

```

{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c124'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 733,
  gender: 'f',
  vampires: 40
},
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c123'),
  name: 'Solnara',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c122'),
  name: 'Rooooooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},
{
  _id: ObjectId('6a1adce5d3a5fd4b1f01c121'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c120'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c11f'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  }
]
learn> █

```


Задание 2.2.4 — Список с первым предпочтением без _id

db.unicorns.find({}, {name: 1, loves: {\$slice: 1}, _id: 0})

```
[learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooodoodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn> █
```

РАЗДЕЛ 2.3 — ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

Задание 2.3.1 — Самки весом 500–700 кг без _id

`db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})`

```
[learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> █
```

Задание 2.3.2 — Самцы от 500 кг, любящие grape И lemon, без _id

`db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}, {_id: 0})`

```
[learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> █
```

Задание 2.3.3 — Единороги без ключа vampires

`db.unicorns.find({vampires: {$exists: false}})`

```
[learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c129'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> █
```

Задание 2.3.4 — Упорядоченный список самцов с первым предпочтением

`db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})`

```
[learn> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn> █
```

РАЗДЕЛ 3.1 — ВЛОЖЕННЫЕ ОБЪЕКТЫ И КУРСОРЫ

Задание 3.1.1 — Коллекция towns

Создаём коллекцию вставив документы:

```
db.towns.insert({name: "Punxsutawney", population: 6200, last_census:
ISODate("2008-01-31"), famous_for: ["phil the groundhog"], mayor: {name:
"Jim Wehrle"}})
```

```
db.towns.insert({name: "New York", population: 22200000, last_census:
ISODate("2009-07-31"), famous_for: ["statue of liberty", "food"], mayor:
{name: "Michael Bloomberg", party: "I"}})
```

```
db.towns.insert({name: "Portland", population: 528000, last_census:
ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "Sam
Adams", party: "D"}})
```

```
learn> db.towns.insert({name: "Punxsutawney", population: 6200, last_census: ISODate("2008-01-31"), famous_for: ["phil
l the groundhog"], mayor: {name: "Jim Wehrle"}})
| db.towns.insert({name: "New York", population: 22200000, last_census: ISODate("2009-07-31"), famous_for: ["statue o
f liberty", "food"], mayor: {name: "Michael Bloomberg", party: "I"}})
| db.towns.insert({name: "Portland", population: 528000, last_census: ISODate("2009-07-20"), famous_for: ["beer", "fo
od"], mayor: {name: "Sam Adams", party: "D"}})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1ade76d3a5fd4b1f01c12d') }
}
learn> █
```

Города с независимыми мэрами (party="I"), только название и мэр:

```
db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
```

```
learn> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> █
```

Беспартийные мэры (нет поля party):

```
db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
```

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> █
```

Задание 3.1.2 — JavaScript-функция и курсор

Функция для вывода самцов:

```
fn = function() { return this.gender === 'm'; }
```

```
db.unicorns.find({$where: fn})
```



```

learn> fn = function() { return this.gender == 'm'; }
[| db.unicorns.find({$where: fn})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c11f'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c121'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c122'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c125'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c126'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c128'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a1adcffd3a5fd4b1f01c12a'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn> █

```

Курсор — первые 2 самца в алфавитном порядке:

```

var cursor = db.unicorns.find( {gender: 'm'}); null;
cursor.sort( {name: 1}).limit(2); null;

```

```
cursor.forEach(function(obj){ print(obj.name); })
```

```
learn> var cursor = db.unicorns.find({gender: 'm'}); null;  
| cursor.sort({name: 1}).limit(2); null;  
[| cursor.forEach(function(obj){ print(obj.name); })
```

```
Dunx
```

```
Horny
```

```
learn> █
```

РАЗДЕЛ 3.2 — АГРЕГИРОВАННЫЕ ЗАПРОСЫ

Задание 3.2.1 — Количество самок весом 500–600 кг

`db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()`

```
[learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
(node:68332) [MONGODB DRIVER] Warning: cursor.count is deprecated and will be removed in the next major version, please use `collection.estimatedDocumentCount` or `collection.countDocuments` instead
(Use `node --trace-warnings ...` to show where the warning was created)
2
learn> █
```

Задание 3.2.2 — Список всех предпочтений

`db.unicorns.distinct("loves")`

```
[learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> █
```

Задание 3.2.3 — Количество особей обоих полов

`db.unicorns.aggregate({"$group": {_id: "$gender", count: {$sum: 1}}})`

```
[learn> db.unicorns.aggregate({"$group": {_id: "$gender", count: {$sum: 1}}})
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn> █
```

РАЗДЕЛ 3.3 — РЕДАКТИРОВАНИЕ ДАННЫХ

Задание 3.3.1 — Сохранить Barny

```
[learn> db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a1adfcfd3a5fd4b1f01c12e')
}
[learn> db.unicorns.find({name: 'Barny'})
[
  {
    _id: ObjectId('6a1adfcfd3a5fd4b1f01c12e'),
    name: 'Barny',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
learn> █
```

Задание 3.3.2 — Обновить Айна (вес 800, vampires 51)

```
learn> db.unicorns.replaceOne(
|   {name: 'Ayna'},
|   {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 800, gender: 'f', vampires: 51}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c124'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn> █
```


Задание 3.3.3 — Raleigh теперь любит redbull

```
[learn> db.unicorns.updateOne({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c126'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> █
```

Задание 3.3.4 — Всем самцам +5 vampires

```
learn> db.unicorns.updateOne({gender: 'm'}, {$inc: {vampires: 5}}, {multi: true})
[
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
]
learn> db.unicorns.find({gender: 'm'})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c11f'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c121'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c122'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c125'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c126'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c128'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  }
]
```

```

{
  _id: ObjectId('6a1adcffd3a5fd4b1f01c12a'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
},
{
  _id: ObjectId('6a1adfcfd3a5fd4b1f01c12e'),
  name: 'Barney',
  loves: [ 'grape' ],
  weight: 340,
  gender: 'm'
}
]
learn>

```

Задание 3.3.5 — Мэр Портланда теперь беспартийный

```

learn> db.towns.updateOne({name: 'Portland'}, {$unset: {"mayor.party": 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a1ade76d3a5fd4b1f01c12d'),
    name: 'Portland',
    population: 528000,
    last_census: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn>

```

Задание 3.3.6 — Pilot теперь любит и шоколад

```

learn> db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c128'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  }
]
learn>

```

Задание 3.3.7 — Aurora теперь любит и сахар, и лимоны

```
[learn> db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c120'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
[learn> ]
```


РАЗДЕЛ 3.4 — УДАЛЕНИЕ ДАННЫХ

Задание 3.4.1

Пересоздаём коллекцию towns

```
db.towns.drop()
```

```
db.towns.insert({name: "Punxsutawney", population: 6200, last_census:
ISODate("2008-01-31"), famous_for: ["phil the groundhog"], mayor: {name:
"Jim Wehrle"}})
```

```
db.towns.insert({name: "New York", population: 22200000, last_census:
ISODate("2009-07-31"), famous_for: ["statue of liberty", "food"], mayor:
{name: "Michael Bloomberg", party: "I"}})
```

```
db.towns.insert({name: "Portland", population: 528000, last_census:
ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "Sam
Adams", party: "D"}})
```

Удаляем беспартийных мэров

```
db.towns.remove({"mayor.party": {$exists: false}})
```

Проверяем коллекцию

```
db.towns.find()
```

```
learn> db.towns.drop()
| db.towns.insert({name: "Punxsutawney", population: 6200, last_census: ISODate("2008-01-31"), famous_for: ["phil the
groundhog"], mayor: {name: "Jim Wehrle"}})
| db.towns.insert({name: "New York", population: 22200000, last_census: ISODate("2009-07-31"), famous_for: ["statue o
f liberty", "food"], mayor: {name: "Michael Bloomberg", party: "I"}})
| db.towns.insert({name: "Portland", population: 528000, last_census: ISODate("2009-07-20"), famous_for: ["beer", "fo
od"], mayor: {name: "Sam Adams", party: "D"}})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1ae184d3a5fd4b1f01c131') }
}
[learn> db.towns.remove({"mayor.party": {$exists: false}})]
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 1 }
[learn> db.towns.find()]
[
  {
    _id: ObjectId('6a1ae184d3a5fd4b1f01c130'),
    name: 'New York',
    population: 22200000,
    last_census: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a1ae184d3a5fd4b1f01c131'),
    name: 'Portland',
    population: 528000,
    last_census: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> █
```

Очищаем коллекцию

```
db.towns.remove({})
```

Проверяем список коллекций

show collections

```
[learn> db.towns.remove({})  
{ acknowledged: true, deletedCount: 2 }  
[learn> show collections  
towns  
unicorns  
learn> █
```

towns пустая, но присутствует в списке

РАЗДЕЛ 4.1 — ССЫЛКИ

Задание 4.1.1

Создаём коллекцию зон обитания единорогов

```
db.habitats.insert({_id: "forest", name: "Enchanted Forest", description: "Magical dark forest with ancient trees"})
```

```
db.habitats.insert({_id: "mountain", name: "Crystal Mountains", description: "High peaks covered in eternal snow"})
```

```
db.habitats.insert({_id: "meadow", name: "Sunlit Meadow", description: "Bright open fields full of flowers"})
```

Добавляем ссылки на зону обитания нескольким единорогам

```
db.unicorns.update({name: 'Horny'}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
```

```
db.unicorns.update({name: 'Aurora'}, {$set: {habitat: {$ref: "habitats", $id: "meadow"}}})
```

```
db.unicorns.update({name: 'Unicrom'}, {$set: {habitat: {$ref: "habitats", $id: "mountain"}}})
```

Проверяем коллекцию единорогов

```
db.unicorns.find({name: 'Horny'})
```

```
learn> db.habitats.insert({_id: "forest", name: "Enchanted Forest", description: "Magical dark forest with ancient trees"})
learn> db.habitats.insert({_id: "forest", name: "Enchanted Forest", description: "Magical dark forest with ancient trees"})
| db.habitats.insert({_id: "mountain", name: "Crystal Mountains", description: "High peaks covered in eternal snow"})

[| db.habitats.insert({_id: "meadow", name: "Sunlit Meadow", description: "Bright open fields full of flowers"})      ]
{ acknowledged: true, insertedIds: { '0': 'meadow' } }
learn> db.unicorns.update({name: 'Horny'}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
| db.unicorns.update({name: 'Aurora'}, {$set: {habitat: {$ref: "habitats", $id: "meadow"}}})
| db.unicorns.update({name: 'Unicrom'}, {$set: {habitat: {$ref: "habitats", $id: "mountain"}}})      ]
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Horny'})      ]
[
  {
    _id: ObjectId('6a1adce5d3a5fd4b1f01c11f'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68,
    habitat: DBRef('habitats', 'forest')
  }
]
learn> █
```

Получим зон обитания Horny

```
[learn> var horny = db.unicorns.findOne({name: 'Horny'}) ]
[learn> db.habitats.findOne({_id: horny.habitat}) ]
{
  _id: 'forest',
  name: 'Enchanted Forest',
  description: 'Magical dark forest with ancient trees'
}
learn> █
```


РАЗДЕЛ 4.2 — ИНДЕКСЫ

Задание 4.2.1

Пересоздаём unicorns с полем dob:

```
db.unicorns.drop()
db.unicorns.insert({name: 'Horny', dob: new Date(1992,2,13,7,47), loves:
['carrot','papaya'], weight: 600, gender: 'm', vampires: 63})
db.unicorns.insert({name: 'Aurora', dob: new Date(1991,0,24,13,0), loves:
['carrot','grape'], weight: 450, gender: 'f', vampires: 43})
db.unicorns.insert({name: 'Unicrom', dob: new Date(1973,1,9,22,10), loves:
['energon','redbull'], weight: 984, gender: 'm', vampires: 182})
db.unicorns.insert({name: 'Roooooodles', dob: new Date(1979,7,18,18,44),
loves: ['apple'], weight: 575, gender: 'm', vampires: 99})
db.unicorns.insert({name: 'Solnara', dob: new Date(1985,6,4,2,1),
loves:['apple','carrot','chocolate'], weight:550, gender:'f', vampires:80})
db.unicorns.insert({name: 'Ayna', dob: new Date(1998,2,7,8,30), loves:
['strawberry','lemon'], weight: 733, gender: 'f', vampires: 40})
db.unicorns.insert({name: 'Kenny', dob: new Date(1997,6,1,10,42), loves:
['grape','lemon'], weight: 690, gender: 'm', vampires: 39})
db.unicorns.insert({name: 'Raleigh', dob: new Date(2005,4,3,0,57), loves:
['apple','sugar'], weight: 421, gender: 'm', vampires: 2})
db.unicorns.insert({name: 'Leia', dob: new Date(2001,9,8,14,53), loves:
['apple','watermelon'], weight: 601, gender: 'f', vampires: 33})
db.unicorns.insert({name: 'Pilot', dob: new Date(1997,2,1,5,3), loves:
['apple','watermelon'], weight: 650, gender: 'm', vampires: 54})
db.unicorns.insert({name: 'Nimue', dob: new Date(1999,11,20,16,15), loves:
['grape','carrot'], weight: 540, gender: 'f'})
db.unicorns.insert({name: 'Dunx', dob: new Date(1976,6,18,18,18), loves:
['grape','watermelon'], weight: 704, gender: 'm', vampires: 165})
```

Проверяем — можно ли создать уникальный индекс по name:

```
db.unicorns.createIndex({"name": 1}, {"unique": true})
```

```
learn> db.unicorns.createIndex({"name": 1}, {"unique": true})
name_1
learn> □
```

Проверяем что уникальность работает — пробуем добавить дубликат:

```
db.unicorns.insert({name: 'Horny', weight: 100, gender: 'm'})
```

```
[learn> db.unicorns.insert({name: 'Horny', weight: 100, gender: 'm'})
Uncaught:
MongoBulkWriteError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Horny" }
Result: BulkWriteResult {
  insertedCount: 0,
  matchedCount: 0,
  modifiedCount: 0,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {},
  insertedIds: {}
}
Write Errors: [
  WriteError {
    err: {
      index: 0,
      code: 11000,
      errmsg: 'E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Horny" }',
      errInfo: undefined,
      op: {
        name: 'Horny',
        weight: 100,
        gender: 'm',
        _id: ObjectId('6a1ae333d3a5fd4b1f01c13e')
      }
    }
  }
]
learn> █
```

РАЗДЕЛ 4.3 — УПРАВЛЕНИЕ ИНДЕКСАМИ

Задание 4.3.1

Получаем все индексы коллекции unicorns:

```
db.unicorns.getIndexes()
```

```
[learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_',
    { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> ]
```

Удаляем индекс по name:

```
db.unicorns.dropIndex("name_1")
```

```
[learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn> ]
```

Пробуем удалить индекс _id:

```
db.unicorns.dropIndex("_id_")
```

```
[learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> ]
```

индекс по _id нельзя удалить

РАЗДЕЛ 4.4 — ПЛАН ЗАПРОСА

Задание 4.4.1

Создаём большую коллекцию (100 000 документов):

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```

```
[learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1ae444d3a5fd4b1f0347de') }
}
learn> █
```

Выбираем последние 4 документа:

```
db.numbers.find().sort({$natural: -1}).limit(4)
```

```
[learn> db.numbers.find().sort({$natural: -1}).limit(4)
[
  { _id: ObjectId('6a1ae444d3a5fd4b1f0347de'), value: 99999 },
  { _id: ObjectId('6a1ae444d3a5fd4b1f0347dd'), value: 99998 },
  { _id: ObjectId('6a1ae444d3a5fd4b1f0347dc'), value: 99997 },
  { _id: ObjectId('6a1ae444d3a5fd4b1f0347db'), value: 99996 }
]
learn> █
```

Анализ плана запроса БЕЗ индекса:

```
db.numbers.explain("executionStats").find({value: {$gt: 99995}})
```

Создаём индекс по value:

```
db.numbers.createIndex({value: 1})
```

Все индексы коллекции:

```
db.numbers.getIndexes()
```

```
[learn> db.numbers.createIndex({value: 1})
value_1
[learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> █
```

Снова выполняем запрос:

```
db.numbers.find().sort({$natural: -1}).limit(4)
```

Анализ плана С индексом:

```
db.numbers.explain("executionStats").find({value: {$gt: 99995}})
```

	Без индекса	С индексом
Тип сканирования	COLLSCAN	IXSCAN
Время (мс)	82	14

Выводы

В ходе лабораторной работы были получены практические навыки установки и работы с документо-ориентированной СУБД MongoDB.

В рамках ЛР 6.1 была установлена СУБД MongoDB, изучены базовые операции: создание и удаление баз данных и коллекций, просмотр статистики, переименование коллекций.

В рамках ЛР 6.2 были получены практические навыки работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Проведено сравнение производительности запросов с индексами и без: запрос без индекса выполнялся за 82 мс с использованием полного сканирования коллекции (COLLSCAN), тогда как после создания индекса время сократилось до 14 мс за счёт индексного сканирования (IXSCAN) — ускорение почти в 6 раз. Это наглядно демонстрирует эффективность индексирования на больших объёмах данных.